```
self = main
```

Inside a class or module definition, self is set to the object of the class or module being defined. This means that instance variables set inside a class or module definition will be available to class or module methods (because self will be the same when the variables are defined and when the methods execute):

```
class Test
  @var = 99
  def self.value_of_var
    @var
  end
end

Test.value_of_var # => 99
```

The fact that self is set to the class during a class definition turns out to be a dramatically elegant decision, but to see why, we'll first need to have a look at singletons.

Defining Singleton Methods

Ruby lets you define methods that are specific to a particular object. These are called *singleton methods*.

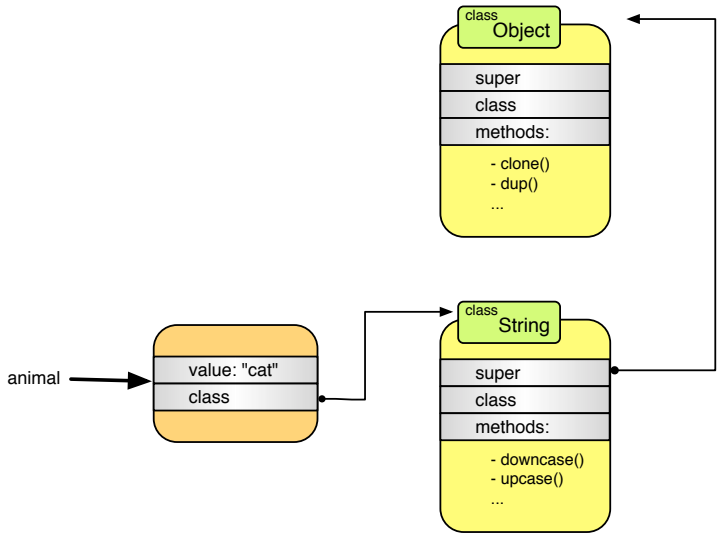
Here's a simple string object and a regular, non-singleton method call:

```
animal = "cat"
puts animal.upcase
```

produces:

```
CAT
```

This call results in the following object structure.



The animal variable points to an object containing (among other things) the value of the string ("cat") and a pointer to the object's class, String.